HANDOFF: Hanafuda Koi-Koi, Giger theme (C + raylib, Windows .exe)

This document is the build contract. Read all of it before your first action. Everything in it is decided. Do not re-open decisions, and do not ask the user to confirm them. If something is truly missing, choose the simplest option, note it in STATUS.md and keep going.

1. Goal

- A single-player Koi-Koi (Hanafuda) game: human vs computer, mouse-driven, delivered as a Windows **.exe** plus an **assets/** folder. The user double-clicks the exe. Nothing else is required.
- Look: Giger biomechanical horror with a light *Alien* (1979) touch: phosphor-green terminal text, faint scanlines, black translucent panels, restrained glow. The supplied art carries the look; the UI stays quiet.
- Play it like any card game on a PC. Grab a card with the mouse, it follows the cursor, drop it on the table. That's it. Section 11 explains it exactly, because earlier attempts failed on this point.

2. Locked decisions

Topic	Decision
Platform	C99 + raylib 5.5, static link. Cross-compiled in the sandbox with MinGW-w64 to KoiKoi.exe (section 5). The same code also builds for Linux so you can run and screenshot it in the sandbox.
Screen	Virtual canvas 1280 x 720, drawn to a RenderTexture, letterboxed into a resizable window (default 1280 x 720, minimum 960 x 540).
Controls	Drag and drop with the mouse (section 11). ESC = pause / back. No click-click play.
Card art	**giger set.png** (2112 x 1144 atlas, 12 x 4 cells of 176 x 286). Final and approved. Do not edit, recolour, slice into files or regenerate it.
Card back	**back.png** (1024 x 1536). Used for the stock and the computer hand.
Backgrounds	harvest.jpg (default), hole.jpg, arctic.jpg, castle.jpg, city.jpg, all 1280 x 720. The chosen one is used on every screen.
Match length	1 to 12 rounds, chosen freely in Options. Default 6.
Difficulty	Easy (default): weak computer, and your playable cards glow. Hard: strong computer, no glow.
Rulesets	Nintendo (default), FudaWiki, KK-ADD-10, chosen in Options. The tables are in section 8.
Saving	Settings and stats only, in text files next to the exe. No mid-match save. The SAVE button writes all files now and shows 'Saved'.
Counting	Finished matches count in stats. RESTART, ABANDON and closing mid-match do not count.
Sound	None needed. You may add it at the end only if all milestones are done.

3. Files the user uploads each session

File	Use
This PDF	The contract.
giger set.png, back.png, 5 backgrounds	Assets. Copy them into the project's assets/ folder (rename atlas to atlas.png).
koikoi_project.zip (from session 2 on)	The project as the last session left it, including STATUS.md. Unzip it and continue.
old hanafuda game code.txt	A finished, working HTA game from an earlier project (10.8 MB, mostly embedded images). **Reference only** for game flow and drag handling (section 13). Never print it whole.
old handoff from previous game.pdf	The spec that produced that HTA game. Optional background reading. **This** document wins wherever they differ.

- Never write, rename or overwrite anything in **uploads/**. Copy from it.
- The workspace does not carry over between chats. The only memory between sessions is the zip the user downloads and uploads again. Section 4 explains how to keep it up to date.

4. How to work: the 15-post budget

A chat session can break after about 15 posts. Plan for that. Every session has to leave a complete, downloadable state behind, even if it ends early.

- **Post 1:** read this PDF and STATUS.md, run the setup script (below), unzip the project, build both targets, run the tests. Then report in at most 5 lines: what builds, what passes and which milestone comes next.
- **One milestone per session** (table below). Don't start the next milestone in the same session unless the current one is fully done.
- **Every 3 posts and at session end:** update STATUS.md (40 lines max: done, in progress, known bugs, next step), zip the project to /home/user/koikoi_project.zip (source, assets, STATUS.md, build script, and the latest KoiKoi.exe) and present the zip. Tell the user to download it.
- Keep chat replies short. Put long output in files. Keep tool output short with head, tail and wc. Never cat big files.
- **Keep source files under about 500 lines.** Small files can be rewritten whole with write_file, which avoids broken partial edits. An earlier main.c was cut off by a failed edit. After every change: compile, check the line count, and run the tests.
- Don't claim something works unless you ran it in this session. Say 'built and tested under Xvfb' or 'not tested yet'.

Milestone	Content	Done when
M1 Engine	cards.c, rules.c, engine.c, ai.c, test_engine.c. No graphics.	3000 random CPU-vs-CPU matches in all 3 rulesets: no crash, 48 cards conserved every step, scoring spot tests pass (section 8).
M2 Table	Board drawing, letterbox, drag and drop, stock reveal, choice overlay, computer turns, Koi-Koi/Shobu overlay, round result.	One full round can be played by mouse under Xvfb (scripted with xdotool). Screenshots look right. The exe builds.
M3 Shell	Start, Options, Stats, Pause, Match result, persistence files, RESTART/ABANDON.	A full 1-round and 3-round match runs end to end. Files are written next to the exe. The exe builds.
M4 Polish	Theme pass (green UI, scanlines, glow), animations, Easy glow, layout tuning. Sound is optional.	Every acceptance item in section 14 is ticked. The user confirms on Windows.

5. Sandbox environment (verified on 11 Oct 2026)

The sandbox **can** compile a Windows exe and **can** run the game with a virtual display, scripted mouse input and screenshots. This was tested with raylib 5.5: a letterboxed drag test window under Xvfb, a drag done with xdotool, a TakeScreenshot, and a MinGW .exe. System packages and /tmp are not kept between sessions, so run this setup in post 1 of every session (about 1 minute):

```
sudo DEBIAN_FRONTEND=noninteractive apt-get update -qq
sudo DEBIAN_FRONTEND=noninteractive apt-get install -y -qq xvfb xdotool mingw-w64 \
  libgl1-mesa-dev libx11-dev libxrandr-dev libxinerama-dev libxcursor-dev libxi-dev > /tmp/apt.log
git clone -q --depth 1 --branch 5.5 https://github.com/raysan5/raylib.git /tmp/raylib
cd /tmp/raylib/src && make -s PLATFORM=PLATFORM_DESKTOP -j8 && mv libraylib.a /tmp/libraylib_linux.a \
 && make -s clean && make -s PLATFORM=PLATFORM_DESKTOP OS=Windows_NT CC=x86_64-w64-mingw32-gcc \
    AR=x86_64-w64-mingw32-ar -j8 && mv libraylib.a /tmp/libraylib_win.a
```

Build commands (put them in build.sh in the project):

```
# Linux (for testing in the sandbox)
gcc -std=c99 -O2 -Wall -Wextra src/*.c -I/tmp/raylib/src /tmp/libraylib_linux.a \
    -lGL -lm -lpthread -ldl -lrt -lX11 -o koikoi_linux
# Windows (the deliverable). -mwindows hides the console window.
x86_64-w64-mingw32-gcc -std=c99 -O2 src/*.c -I/tmp/raylib/src /tmp/libraylib_win.a \
    -lopengl32 -lgdi32 -lwinmm -static -mwindows -o dist/KoiKoi.exe && cp -r assets dist/
# Engine tests (no graphics)
gcc -std=c99 -O2 -Wall -Wextra -Isrc tests/test_engine.c src/cards.c src/rules.c src/engine.c src/ai.c \
    -o test_engine && ./test_engine
```

GUI test under Xvfb. With no window manager, the window sits at (0,0), so xdotool screen coordinates equal window coordinates:

```
Xvfb :99 -screen 0 1280x720x24 & export DISPLAY=:99
./koikoi_linux --seed 7 --shot-frame 600 > run.log &      # game exits after the screenshot
xdotool mousemove 520 630 mousedown 1; sleep 0.2
xdotool mousemove 600 400; sleep 0.2; xdotool mousemove 700 330; sleep 0.2; xdotool mouseup 1
```

The game must support **--seed N** (repeatable deals), **--shot-frame N** (TakeScreenshot, then exit) and **--window WxH** (also test a 1000 x 700 window to check the letterbox maths). It must also print one log line per input event and per engine action, e.g. `PRESS hand[3]=05-R`, `DROP centre=(712,330) -> CAPTURE 05-R+05-P1`. Check the log and look at the screenshot (convert it to JPG before viewing it).

6. Architecture (decided)

File	Responsibility
src/cards.c/.h	CARDS[48] table: id 0..47 = (month-1)*4 + slot, month, type, flags (rain, cup, poetry, blue), name. cards_validate().
src/rules.c/.h	Ruleset tables and score_captures(ruleset, cards[], n) -> yaku list + base. Pure functions.
src/engine.c/.h	Game state and phase machine (below). Deal, misdeal, lucky hands, hand play, stock draw, Koi/Shobu, settlement. No raylib include.
src/ai.c/.h	Computer choices from public information only: own hand, field, both capture piles, stock count.
src/ui_*.c	raylib only: ui_main.c (loop, screens, letterbox), ui_board.c (drawing), ui_input.c (mouse / drag), ui_anim.c (tweens), ui_screens.c (menus, overlays).
src/storage.c/.h	settings.txt, stats.txt, history.txt, stats_report.txt in GetApplicationDirectory().
tests/test_engine.c	Random matches plus fixed scoring cases. Runs without graphics.

- **The engine is the only owner of rules and card zones.** The UI never moves a card between zones. It asks the engine, then animates the result. Animations are only for show; they never decide anything.
- Card zones: hand[2], field, stock, captured[2], pending (the revealed stock card). Each of the 48 ids is in exactly one zone. Check this after every engine call in debug builds and in tests.

Phase machine (engine), with the UI calls that move it forward:

```
DEAL -> (misdeal: redeal same dealer) -> LUCKY_CHECK -> (lucky: ROUND_END) -> TURN_HAND
TURN_HAND      human: engine_play_hand(g, card, target_or_-1)   cpu: ai_choose_hand()
               -> needs choice (2 matches, no target given) -> HAND_CHOICE (can cancel back to TURN_HAND)
               -> resolved -> TURN_DRAW
TURN_DRAW      engine_reveal_stock(g) -> 0/1/3 matches resolve at once; 2 matches -> DRAW_CHOICE (no cancel)
CHECK_YAKU     base rose? -> DECIDE (offer KOI-KOI / SHOBU) else next player / ROUND_END on exhaustion
DECIDE         engine_decide(g, KOI|SHOBU) -> next player | ROUND_END
ROUND_END      settlement written to ledger -> next round or MATCH_END
```

The UI pumps the engine: when no animation is running and the phase needs no human input, take the next step (reveal the stock, run the computer move after its delay, and so on). Input is locked while animations run or while it is the computer's turn.

7a. Rules: deal and turn

- 48 cards, 12 months x 4. **Matching is by month only.** Art, type and names never decide a match. Lightning and the Sake Cup are not wild.
- First dealer of a match: each side draws one card; the earlier month deals (same month: redraw). After that, the round winner deals, and on a draw the dealer stays. Dealer plays first.
- Deal: 8 to each hand, 8 face up to the field, 24 to the stock. Shuffle with Fisher-Yates and a seeded PRNG (xorshift32), kept separate from any random numbers used by the AI or animations.
- Misdeal: if the field holds all 4 cards of a month, or exactly four pairs, redeal (same dealer, round not counted). Show 'Misdeal - redealing'.
- Lucky hands, checked once after a valid deal: Teshi (all 4 of a month in hand) or Kuttsuki (four pairs, 2-2-2-2) = 6 points to that player, round over, no multipliers. Both players lucky = 0-0 draw.
- **Turn = hand stage, then stock stage, always both.** Hand: play one card. 0 field matches = it goes onto the field. 1 = capture that pair. 2 = capture the one the player picks. 3 = capture all four. You may play a non-matching card even if another card would match, but a card that matches must capture.
- Stock: reveal the top card and apply the same 0/1/2/3 rule to the updated field (it can take the card just played). With 2 matches the player must pick; no cancel. The stock card never goes into a hand.
- Clearing the field is not special. It doesn't end the round and gives no extra turn.

7b. Rules: Koi-Koi, round end, match

- After both stages, score the player's captures. If the base is **higher than that player's last base** this round, offer KOI-KOI or SHOBU. If not, no prompt; play passes on. Several new yaku in one turn = one decision.
- SHOBU: the round ends and this player scores. KOI-KOI: play continues; the player's koi flag stays set for the rest of the round. Both players may call.
- If the deciding player has no cards left in hand, KOI-KOI is not offered. SHOBU is applied automatically.
- Koi-Koi limit: Nintendo allows one call per player per round; after that, the next base increase is an automatic SHOBU. FudaWiki and KK-ADD-10 allow unlimited calls. (FudaWiki's take-back of a call is not implemented.)
- Exhaustion: when both hands are empty and nobody called SHOBU, the round is a 0-0 draw and the dealer stays. That is at most 16 turns; 8 stock cards stay unused. Do not stop when only the first hand is empty.
- Award = base x (base >= 7 ? 2 : 1) x (opponent called Koi-Koi this round ? 2 : 1). The winner gets the award, the loser gets 0. Your own Koi-Koi never doubles your own score. The opponent's flag doubles only once.
- Match = the chosen number of counted rounds (1-12). Higher total wins; equal totals = draw. Misdeals are not counted as rounds.

Base	Opponent koi?	Award
1	no	1
5	yes	10
6	no	6
7	no	14
7	yes	28
12	no	24

8. Scoring (score only captured cards)

Yaku	Condition	Nintendo	FudaWiki	KK-ADD-10
Goko	all 5 Brights	10	15	10
Shiko	4 Brights, no Rain	8	8	8
Ame-shiko	Rain + 3 other Brights	7	7	7
Sanko	3 Brights, no Rain	5	6	5
Hanami-zake	Curtain 03-B + Sake Cup 09-A	5	5	5
Tsukimi-zake	Moon 08-B + Sake Cup 09-A	5	5	5
Ino-shika-cho	Butterflies 06-A + Boar 07-A + Deer 10-A	5, +1 per other animal	5, +1 per other animal	5 (fixed)
Akatan	3 poetry ribbons 01/02/03	5, +1 per other ribbon	5, +1 per other ribbon	5 (fixed)
Aotan	3 blue ribbons 06/09/10	5, +1 per other ribbon	5, +1 per other ribbon	5 (fixed)
Akatan + Aotan	all 6 above	10 (replaces both), +1 per other ribbon	same as Nintendo	no extra yaku (5 + 5)
Tane	n animals (Cup counts), n >= 5	n-4 *	n-4 *	n-4 (stacks)
Tanzaku	n ribbons (all 10 kinds), n >= 5	n-4 *	n-4 *	n-4 (stacks)
Kasu	k plains, k >= 10	k-9	k-9	k-9, Sake Cup adds 1 to k

- **Bright family: exactly one counts**, the highest that applies. 5 Brights = Goko only (not 10+8+7+5). 3 Brights that include Rain = 0.
- Viewings are independent of the Brights and of each other: Curtain + Moon + Cup = 10. Rain never cancels them.
- * Nintendo / FudaWiki: Ino-shika-cho's '+1 per other animal' **replaces** Tane when both apply (score the higher one; the set is always higher). The same goes for the ribbon sets vs Tanzaku. A card is never counted twice.
- KK-ADD-10: the fixed sets and the counts stack. Trio + 2 other animals = 5 + 1 = 6. 3 poetry + 3 blue = 5 + 5 + Tanzaku (6-4=2) = 12. 9 plains + Cup = Kasu 1.
- Base = one Bright yaku (or 0) + every other yaku that applies. The Sake Cup is one physical card but can count in Tane, both viewings and (KK-ADD-10) Kasu at the same time.
- Required test cases (M1): Goko 10/15/10; Rain + 3 = 7; Rain + 2 = 0; Curtain + Moon + Cup = 10; KK Trio + 2 = 6; KK all 6 ribbons = 12; KK 9 plains + Cup = 1; Nintendo all 6 ribbons = 10; Nintendo Trio + 2 = 7 (not 7 + 1); the award table above.

9a. The deck and the atlas (checked against giger set.png)

Atlas cell (column c, row r), both 0-based: x = c*176, y = r*286, w = 176, h = 286. Column = month - 1 (January on the left). Row = slot. **Card id = c*4 + r**, which is also the canonical deck order. Codes: B Bright, A Animal, RP poetry ribbon (red with writing), RB blue ribbon, R plain red ribbon, P plain.

Col	Month	Row 0	Row 1	Row 2	Row 3
0	Jan Pine	B crane/sun	RP	P	P
1	Feb Plum	A warbler	RP	P	P
2	Mar Cherry	B curtain	RP	P	P
3	Apr Wisteria	A cuckoo	R	P	P
4	May Iris	A bridge	R	P	P
5	Jun Peony	A butterflies	RB	P	P
6	Jul Clover	A boar	R	P	P
7	Aug Pampas	B moon	A geese	P	P
8	Sep Chrys.	A sake cup	RB	P	P
9	Oct Maple	A deer	RB	P	P
10	Nov Willow	B rain	A swallow	R	P lightning
11	Dec Paulownia	B phoenix	P	P	P

- Counts (cards_validate() must check them at startup): 5 Brights (01, 03, 08, 11, 12; Rain = 11-B only), 9 Animals, 10 Ribbons (3 RP, 3 RB, 4 R), 24 Plains. August has no ribbon. November has one plain (Lightning, not wild). December has three plains.
- 05-A (bridge) and 10-A (deer) have figures on them but are Animals, not Brights. Writing and seals on a card (e.g. the gold 'fuku' seal on two December plains) are art only.
- Dealer-draw rank, used only for the first-dealer tie-break: Bright 20, Animal 10, Ribbon 5, Plain 1.

9b. Asset handling

- Load the atlas once as a Texture2D. Call GenTextureMipmaps and TEXTURE_FILTER_TRILINEAR, because cards are drawn at 40-50% of the cell size and look jagged without it. Draw with DrawTexturePro(atlas, cellRect, destRect, ...). Never stretch: every card box below keeps the 176:286 ratio.
- back.png is 1024 x 1536 (ratio 0.667, cards are 0.615). At load time: ImageCrop to the centre 945 x 1536, ImageResize to 176 x 286, then make it a texture. Do this in memory only; never change the file.
- Backgrounds are exactly 1280 x 720, so draw them 1:1 on the canvas. Then lay a black rectangle at about 35% alpha over the game table so the cards stand out. Options order: 0 harvest (alien over red-lit wasteland), 1 hole (organic city with an egg), 2 arctic (dark pipes, teal light), 3 castle (moon over a biomech landscape), 4 city (alien head wall with faces).
- Paths: build all asset and save paths from GetApplicationDirectory(). A double-click on Windows can start the exe with a different working folder.
- Font: raylib's default font, scaled in whole steps (10, 20, 30 px). Its blocky style suits the 'ship computer' look. No network fonts.

9c. Theme values

Element	Value
Text / accents	phosphor green #5CFF8A; dim text #2E9E55; warnings amber #FFB000; error red #FF4040
Panels / buttons	black at 70% alpha, 1 px green border. Hover: border brighter plus 15% green fill. Disabled: 40% alpha
Scanlines	after drawing the whole canvas, a 1 px black line every 3 px at about 12% alpha. Turn off on the card-choice overlay if it hurts readability
Glow	2-3 px green outline (DrawRectangleLinesEx, alpha pulses 0.5-1.0 over 1 s). Used for Easy hints and choice targets only

10a. Table layout (virtual 1280 x 720, decided)

Zone	Rectangle / rule
Left column	x 12..312. CPU info bar y 12 (name, total score, DEALER tag). CPU captures y 48..304. Your captures y 420..676. Your info bar y 680.
Capture lanes	4 lanes per player, top to bottom: Brights, Animals, Ribbons, Plains. Lane height 62, mini card 36 x 58. A label with count at the left (e.g. 'P 9+cup'). Cards fan with step = min(40, 236/(n-1)); never cropped, never wrapped.
CPU hand	Card backs 56 x 91, y = 12, step 62, centred on x = 710. Face down, not interactive.
Stock	x 330, y 262, 76 x 124 (back), stack offset 1 px per 4 cards, remaining count below. Not a drop target.
Field	Rectangle F = x 430, y 132, w 650, h 400. Cards 76 x 124 in 2 rows (y 160 and y 318), sorted by month, then id. Columns = max(7, ceil(n/2)), step = min(88, 640/columns). Centred in F.
Your hand	Cards 88 x 143, y = 566, step 96, centred on x = 710. A hovered card lifts 14 px (visual only).
Right column	x 1100..1268. COMBOS panel y 12..500: live yaku with points for YOU and CPU, current base and koi flags. Round 'R 2/6' and ruleset name. PAUSE button y 560, RESTART y 610 (168 x 40 each).
Status line	One line of text, centred at y 540: 'Your turn - drag a card onto the table', 'CPU is thinking...', etc.

You can shift any number by up to about 20 px if something overlaps. These rules must still hold: no zone overlaps another, the field never touches the hands, and nothing is drawn outside 1280 x 720.

10b. Animations (visual only, input locked while they run)

Event	Time
Drag follow	0 ms: the card is exactly under the cursor every frame (grab offset kept). No easing or lag.
Drop onto field / capture move	200 ms ease-out to the target, then 250 ms into the owner's capture lane
Far drop (return to hand)	180 ms ease-out back to its exact hand slot
Stock reveal	150 ms flip at the stock, short pause (300 ms), then resolve
Field / hand reflow	150 ms; stable order
Computer move	think 500-900 ms, then card flies 300 ms from its hand slot

11. Mouse and drag and drop: exact contract

Read this twice. Earlier attempts broke here, almost always for one of the reasons in 11c. The behaviour you want is what any digital card game does.

11a. Behaviour

- Press the left button on one of **your** hand cards while it is your hand stage -> that card is picked up. It is drawn on top of everything and follows the cursor, keeping the point where you grabbed it. Its hand slot stays reserved (the other hand cards do not move).
- While dragging (Easy only): matching field cards glow.
- Release: compute the dragged card's **centre** c = mouse - grab + (w/2, h/2), all in virtual coordinates.
  1. If c is not inside the field rectangle F grown by 24 px on every side: the card glides back to its slot. Nothing else happens.
  2. Inside: m = number of field cards of the same month. m = 0: the card is placed on the field. m = 1: capture that card (no matter where in F it was dropped). m = 3: capture all three.
  3. m = 2: if c lies inside exactly one of the two matching cards' rectangles (grown by 10 px), take that one. Otherwise open the **choice overlay**: both candidates drawn large (176 x 286) side by side. Click one to take it; CANCEL puts the card back in your hand.
- Hovering never changes the game. Pressing on anything else (field, stock, CPU hand, empty space) does nothing during play.
- Stock with 2 matches on your turn: the same overlay without CANCEL.
- Buttons: highlight on hover, act on **release** over the same button that got the press.
- If the window loses focus or ESC is pressed mid-drag: cancel the drag (card glides back), then pause.

11b. Reference implementation (raylib; verified pattern for the letterbox maths)

```
typedef struct { float scale, ox, oy; } Letterbox;
static Letterbox LB(void){ float sw=GetScreenWidth(), sh=GetScreenHeight();
  float s = fminf(sw/1280.0f, sh/720.0f); return (Letterbox){ s, (sw-1280*s)/2, (sh-720*s)/2 }; }
static Vector2 VMouse(Letterbox l){ Vector2 m=GetMousePosition();          /* ONE conversion, ONE place */
  return (Vector2){ (m.x-l.ox)/l.scale, (m.y-l.oy)/l.scale }; }

/* each frame, in this order */
Letterbox l = LB(); Vector2 vm = VMouse(l);           /* used by ALL hit tests: cards, buttons, overlays */
if (!drag.on && IsMouseButtonPressed(MOUSE_BUTTON_LEFT) && input_unlocked && phase==TURN_HAND)
   for (int i=hand_n-1;i>=0;i--)                        /* topmost first = reverse draw order */
      if (CheckCollisionPointRec(vm, hand_rect(i))) { drag.on=1; drag.idx=i;
         drag.grab=(Vector2){vm.x-hand_rect(i).x, vm.y-hand_rect(i).y}; break; }
if (drag.on) { drag.pos=(Vector2){vm.x-drag.grab.x, vm.y-drag.grab.y}; }      /* no lerp */
if (drag.on && IsMouseButtonReleased(MOUSE_BUTTON_LEFT)) { drag.on=0;
   Vector2 c={drag.pos.x+HAND_W/2, drag.pos.y+HAND_H/2}; resolve_drop(drag.idx, c); }

BeginTextureMode(canvas); draw_board(); if (drag.on) draw_card_at(drag.card, drag.pos); EndTextureMode();
BeginDrawing(); ClearBackground(BLACK);                       /* the -720 flips the RenderTexture */
DrawTexturePro(canvas.texture,(Rectangle){0,0,1280,-720},
               (Rectangle){l.ox,l.oy,1280*l.scale,720*l.scale},(Vector2){0},0,WHITE); EndDrawing();
```

11c. Known causes of broken mouse handling (check every one)

- Hit tests using raw GetMousePosition() (window pixels) while cards are in canvas coordinates. Every hit test must use vm.
- Converting twice: using SetMouseOffset/SetMouseScale **and** manual maths. Use only the VMouse() function above.
- Forgetting to flip the RenderTexture (source height -720), so everything is drawn upside down and clicks seem to hit the wrong card.
- FLAG_WINDOW_HIGHDPI: don't set it. On Windows display scaling (125%/150%) it makes the screen and render sizes differ. Use GetScreenWidth/Height for the letterbox.
- Taking the card out of the hand on press, so the hand reflows under the cursor and the next hit test picks a different card. The card leaves the hand only when the engine commits the play.
- Testing the drop with an eased or animated position instead of mouse - grab. The old HTA game had this right (section 13).
- Using IsMouseButtonDown for a press, which fires every frame and re-grabs. Use Pressed to start, Released to end.
- Hit testing in draw order instead of reverse draw order, so the card underneath gets picked.
- Not locking input during animations or the computer turn, so a second drop makes a double play.
- Logic inside drawing code. Do input, then the engine, then animation, then drawing; drawing changes no state.

12. Screens

Screen	Content
Start	Title 'HANAFUDA // KOI-KOI' in green terminal style over the chosen background. Buttons: START GAME, OPTIONS, STATS, SAVE, QUIT. SAVE writes all files and shows 'Saved' for 2 s.
Options	Rounds [-] n [+] (1-12). Difficulty Easy/Hard. Ruleset Nintendo/FudaWiki/KK-ADD-10. Background < name > with a small preview. BACK saves settings.txt and returns.
Game table	Section 10a. ESC or PAUSE opens Pause. RESTART asks 'Restart match? (not counted)' YES/NO.
Pause	Darkened table. RESUME, RESTART (new match, not counted), ABANDON (back to Start, not counted). The computer timer and animations stop while paused.
Koi-Koi decision	Shows your new yaku with mini cards, the base, and 'If you stop now: base x multipliers = award'. Buttons KOI-KOI and SHOBU. When the computer calls Koi-Koi: a 1.5 s banner 'CPU: KOI-KOI'.
Round result	Winner, each yaku with points and mini cards, base, x2 (7+), x2 (opponent koi), award, running score. Draws show the reason (exhaustion / both lucky). Button NEXT ROUND (or RESULTS after the last round).
Match result	Final score, WIN / LOSS / DRAW, the round ledger (round, winner, base, multipliers, award). Buttons NEW MATCH and MENU. The match is saved to stats here.
Stats	Matches played / won / lost / drawn, win rate, rounds won, total points for/against, best award, koi calls, Easy/Hard split, current and best streak, last 10 matches. Buttons SAVE LOG (writes stats_report.txt), RESET (with confirm), BACK.
Too small	If the window is smaller than 960 x 540: show 'Enlarge the window' and accept no input. Never crop the table.

12a. Files (next to the exe, plain text key=value)

- settings.txt: rounds=6, difficulty=easy, ruleset=nintendo, background=0. If a file is missing or broken, use the defaults; never crash.
- stats.txt: the counters from the Stats screen. history.txt: one line per finished match `2026-10-11 21:14;rounds=6;nintendo;easy;you=23;cpu=11;WIN`.
- stats_report.txt: the Stats screen as readable text. Write it on match end, SAVE and SAVE LOG. Write files safely: write a .tmp file, then rename it over the old one.

12b. Computer player (public information only)

- Card value: Bright 20, Animal 10, Ribbon 5, Plain 1, plus 15 if taking it raises the computer's base, plus 6 if it blocks a set the human is one card from completing (judged from the human's captures).
- **Hard:** for each hand card, score = value of everything it captures, or for a discard, -(its value) minus 4 if its month still has unseen cards (the human could take it). Play the best. With 2 targets, take the higher value. Koi-Koi only if base < 7, computer hand >= 4 and human base = 0; otherwise SHOBU.
- **Easy:** 30% of the time play a random card; otherwise the first legal capture, else a random discard. Random pick when there are 2 targets. Koi-Koi 50% of the time while the hand has >= 3 cards.
- The AI must never read the human hand or the stock order. Test this: swap the human hand for different cards; the AI's choice must not change.

13. Using the old HTA game (old hanafuda game code.txt)

It is a finished and working Koi-Koi in one HTML/JS file ('KonoSuba Koi-Koi build 136', 1600 x 900 stage). Its theme, characters, shop, quests and 'lab' are not relevant. It is useful as proof of a working **turn flow** and **drag handling**. Read it with the base64 removed (line numbers stay the same):

```
python3 -c "import re;s=open('uploads/old hanafuda game code.txt',encoding='utf-8',errors='replace').read();\
open('/tmp/old.js','w').write(re.sub(r'[A-Za-z0-9+/=]{300,}','B64',s))"; sed -n 1855,1925p /tmp/old.js | cut -c1-200
```

Lines	What	Take from it
1075	UI.resize	Stage scaling; input = client / scale. It stretches without letterboxing; we letterbox instead (section 11b).
1224-1248	evalYaku	Do **not** copy. It double-counts (Chofuku + Akatan + Aotan + Tan). Use section 8.
1323-1360	dealRound	Deal order. Its misdeal check only tests 4-of-a-month; ours also tests four pairs.
1524-1556	doCapture / doPlace	Hand-stage resolution and animation into the capture pile.
1560-1590	deckFlip / chooseOpt	Stock reveal, then 0/1/3 resolve, or 2 -> human choice / AI pick.
1612-1658	afterCaptures / nextTurn	Score -> Koi prompt only if points rose; auto-SHOBU with an empty hand; skip a player whose hand is empty; draw when both are empty.
1660-1680	aiTurn	Computer move with a thinking delay; Easy = 30% random card.
1708-1735	roundWin / roundDraw	Multipliers (opponent koi x2, 7+ x2) and the result overlay.
1855-1924	dragStart / dragMove / dragEnd	The drag that worked: grab offset kept, hit test on the **intended** position (mouse - offset), not the eased one; snap back if not on the table.

Differences on purpose: we drop matching cards anywhere on the table (it required dropping onto the match), the drag follows with no lag (it used 45% easing per frame), and the scoring is section 8, not its evalYaku.

14. Acceptance checklist (M4 is done when all pass)

#	Check	How
1	test_engine: 3000 matches x 3 rulesets, 48 cards conserved, all scoring cases of section 8 pass	sandbox
2	Drag: card follows the cursor exactly; far drop returns; drop at 23 px outside F commits, 25 px returns; dropping on stock, lanes, CPU hand or panels never commits	xdotool + log
3	The same drag tests pass in a 1000 x 700 window (letterbox bars) and a 1600 x 900 window	xdotool
4	2-match hand choice (direct drop and overlay + CANCEL); 2-match stock choice (no cancel); 3-match takes all four; a deliberate non-matching discard works	seeded deals
5	Koi-Koi prompt only when the base rises; Nintendo 1-call limit; counter-koi x2; 7+ x2; empty hand = auto-SHOBU; exhaustion = 0-0 draw, dealer stays	tests + play
6	Misdeal redeals; Teshi/Kuttsuki give 6; both lucky = 0-0	forced seeds
7	Options persist; 1, 6 and 12 round matches finish; stats and history update; RESTART/ABANDON are not counted	play
8	Easy glows and plays weak; Hard has no glow and plays well; AI ignores the hidden human hand (swap test)	tests
9	Every screen uses the chosen background; green UI; scanlines; nothing overlaps at 1280 x 720	screenshots
10	KoiKoi.exe + assets/ in a zip; runs by double-click on Windows; files appear next to the exe	user on Windows

15. First session plan (no project zip yet)

- Post 1: run the section 5 setup, create koikoi/ with src/, tests/, assets/ (copy from uploads/), build.sh and STATUS.md. Report in 5 lines.
- Posts 2-6: M1. Write cards.c, rules.c, engine.c, ai.c and test_engine.c. Compile and run after each file.
- Post 7: zip and present. Posts 8-13: start M2 (letterbox + board drawing first, then drag, then stock/choice, then computer turn). Take an Xvfb screenshot after each step.
- Last post (or whenever things start failing): update STATUS.md, zip, present, and tell the user to download it.

16. STATUS.md template (keep it at 40 lines max)

```
# STATUS - session N - date
Milestone: M2 (in progress)
Builds: linux OK | exe OK | test_engine OK (3000x3, 0 failures)
Done this session: - board drawing + letterbox  - drag & drop (Xvfb tested, 1280x720 + 1000x700)
In progress: stock 2-match overlay (ui_screens.c, draw done, input not wired)
Known bugs: - capture lane label overlaps at 20+ plains
Next step: wire overlay input, then computer turn (ai_choose_hand + delay)
Deviations from handoff: none   (or: list them, one line each)
```

17. Short glossary

Term	Meaning
Yaku	A scoring combination of captured cards.
Base	Sum of the yaku points before multipliers.
Koi-Koi	Keep playing for more points (risky: the opponent's award doubles if they win the round).
Shobu	Stop and score the round now.
Teshi / Kuttsuki	Lucky opening hands (4 of a month / four pairs) = 6 points.
Field / stock	Face-up table cards / face-down draw pile.
Bright, Animal, Ribbon, Plain	The four card types (20 / 10 / 5 / 1 by tradition; only used for the dealer tie-break).

Summary for the agent: build a normal, solid card game. The rules are in sections 7-8, the art is final, the mouse behaviour is section 11, and you can test everything yourself in the sandbox (section 5). Keep files small, test after every change, and leave a downloadable zip behind every few posts.